DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

(Artificial Intelligence & Machine Learning)

Academic Year:	2026-27	**Name of Student:**	
Semester:	V	**Student ID:**	
Class / Branch:	TE – CSE (AI&ML)	**Roll No:**	
Subject:	WMD Lab	**Date of Submission:**	

Experiment No. 09

Lab Outcome: Apply microservices, REST APIs, and deployment techniques for scalable applications.

Aim: Deploy Spring Boot app to a cloud platform (Render/Heroku) or local server.

Software used: Java JDK17 or above, Spring Boot, Maven, Git & GitHub Account, Render Account

Theory: -

1. What is Application Deployment?

Deployment is the process of taking an application from a developer's own machine and making it available for other people to actually use — either on a local network, or, more commonly, on the public internet via a cloud platform. A Spring Boot application that only "works on my laptop" has no real value to end users until it has been deployed somewhere they can reach it.

	Local Deployment	Cloud Deployment
Where it runs	The developer's own computer	A remote server managed by a cloud provider (e.g. Render)
Who can access it	Only the developer (via localhost)	Anyone with the public URL, from anywhere
Purpose in this	Verifies the application builds and runs	Demonstrates a real, internet-

	Local Deployment	Cloud Deployment
experiment	correctly before deployment	accessible deployment

2. Maven and the Build Lifecycle

Maven is a build automation and dependency management tool for Java projects. Instead of manually downloading libraries such as Spring Web, a project simply declares them in a file called pom.xml, and Maven downloads and manages them automatically. Maven also defines a standard build lifecycle — a sequence of phases such as compile, test, and package — that transforms raw source code into a runnable artifact.

Most Spring Initializr projects also include the Maven Wrapper (mvnw / mvnw.cmd) — a small script committed alongside the project that downloads the exact correct Maven version automatically. This is why the build command in this experiment is .\mvnw rather than a plain mvn: it guarantees the same Maven version is used regardless of what is (or is not) already installed on the machine.

3. Anatomy of a Spring Boot Application

Every Spring Boot application needs exactly one class annotated with @SpringBootApplication and a main() method that calls SpringApplication.run(...). This class is the entry point: it is what actually starts the embedded web server and brings the rest of the application to life. The Implementation section below explains this class, DemoApplication.java, and its annotations in detail.

4. Version Control with Git and GitHub

Git is a version control system that tracks changes to a project's source code over time. GitHub is a cloud-hosted platform for storing Git repositories online. Cloud platforms such as Render do not accept a ZIP file or a folder upload for deployment — instead, they connect directly to a GitHub repository and rebuild the application automatically whenever new code is pushed to it. This is why, in Part B of this experiment, the project must first be committed to Git and pushed to GitHub before Render can deploy it at all.

A Personal Access Token (PAT) is used in place of a GitHub password when pushing code from the command line. GitHub no longer accepts plain passwords for this purpose, so a PAT — a

long, revocable, scope-limited credential — is generated once and used instead.

5. Containerization with Docker

Docker packages an application together with everything it needs to run — the operating system libraries, the Java runtime, and the compiled application itself — into a single, portable unit called a container image. Instead of a cloud platform having to guess how to build and run an arbitrary Java project, a Dockerfile gives it an exact, repeatable recipe to follow.

A multi-stage Dockerfile (used in this experiment) builds the application in two separate stages: a build stage, which contains Maven and the full JDK and is used only to compile the project into a .jar file, and a run stage, which starts from a much smaller image containing only the Java Runtime Environment (JRE) and copies in just the finished .jar file. This keeps the final deployed image small and secure, since none of the build tools are shipped to production.

6. Platform-as-a-Service (PaaS) Deployment

Render is a Platform-as-a-Service (PaaS): a cloud provider that manages the underlying servers, networking, and infrastructure, so a developer only has to supply the application code (and, in this case, a Dockerfile). When Docker is selected as the deployment method, Render automatically detects and uses the Dockerfile committed to the repository to build and run the container — no manual build or start commands need to be configured.

7. Environment-Based Configuration

Cloud platforms typically assign a web application's port dynamically at startup, through an environment variable named PORT, rather than always using a fixed port such as 8080. The property server.port=${PORT:8080} tells Spring Boot: "listen on whatever port the PORT environment variable specifies; if that variable is not set (as on a local machine), fall back to 8080 instead." This single line lets the exact same application run correctly both locally and on Render, without any code changes between the two environments.

Conclusion: -

Thus, we have demonstrated and Deploy Spring Boot app to a cloud platform (Render/Heroku) or local server.